

Codes

Python Code (Exported from IPYNB)

```
# %%
# Import necessary libraries
import pandas as pd
import numpy as np
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor
import os
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error
import matplotlib.pyplot as plt

# %%
# Define base directory for datasets
BASE = os.getcwd()
DATASETS_DIR = os.path.join(BASE, "datasets")

# %%
# --- Part (a): NSS068 Dataset Analysis ---

# Load the NSS068 dataset, handling potential mixed types
df_001 = pd.read_csv(os.path.join(DATASETS_DIR, "NSS068.csv"), low_memory=False)

# %%
# Display unique states to verify data loading
print(df_001['state_1'].unique())

# %%
# Filter data for "MEG" state and select relevant columns for analysis
df_001 = df_001[df_001['state_1'] == "MEG"]
df_001 = df_001[['foodtotal_q', 'MPCE_MRP', 'Age', 'Meals_At_Home',
'Possess_ration_card', 'Education', 'No_of_Meals_per_day']]

# %%
# Display DataFrame information, including non-null counts and data types
df_001.info()

# %%
# Define a function to impute missing values with the mean for specified columns
def impute_with_mean(df):
    cols_to_impute = ['Meals_At_Home', 'No_of_Meals_per_day']
    for col in cols_to_impute:
        # Fill NaN values with the mean of the respective column
        df[col] = df[col].fillna(df[col].mean())
    return df
```

```
# Apply the imputation function to the DataFrame
df_001 = impute_with_mean(df_001)

# %%
# Replace infinite values (positive or negative) with NaN
df_001 = df_001.replace([np.inf, -np.inf], np.nan)

# Check for any remaining missing values after imputation and replacement
# This step also confirms the previous dropna was redundant if imputation handled
# all NaNs
print(df_001.isna().sum())

# %%
# Prepare data for OLS regression: 'foodtotal_q' as dependent variable (y)
# and other selected columns as independent variables (X), adding a constant.
y = df_001['foodtotal_q']
X = sm.add_constant(df_001.drop(['foodtotal_q'], axis=1).copy())

# %%
# Create and fit the Ordinary Least Squares (OLS) regression model
model = sm.OLS(y, X)
outputs = model.fit()

# %%
# Print the summary of the regression results
print(outputs.summary())

# %%
# Calculate Variance Inflation Factor (VIF) to check for multicollinearity
vif = pd.DataFrame()
vif['feature'] = X.columns
vif['VIF'] = [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
print(vif)

# %%
# Extract and print the coefficients of the fitted model
coeff = outputs.params
print(coeff)

# %%
# Construct and print the regression equation
eq = f"y = {round(coeff.iloc[0], 2)}"
for i in range(1, len(coeff)):
    eq += f" + {round(coeff.iloc[i], 6)}*x{i}"
print(eq)

# %%
# Print the first row values of the selected features and the target variable
# This is for inspection and can be removed if not needed for further analysis.
print(df_001['MPCE_MRP'].head(1).values[0])
print(df_001['Age'].head(1).values[0])
print(df_001['Meals_At_Home'].head(1).values[0])
print(df_001['Possess_ration_card'].head(1).values[0])
print(df_001['Education'].head(1).values[0])
```

```

print(df_001['No_of_Meals_per_day'].head(1).values[0])
print(df_001['foodtotal_q'].head(1).values[0])

# %%
# Delete the DataFrame to free up memory
del df_001

# %%
# --- Part (b): IPL Player Performance and Salary Analysis ---

# Load IPL performance and salary datasets
perf_df = pd.read_csv(os.path.join(DATASETS_DIR, 'IPL_ball_by_ball_updated till
2024.csv'), low_memory=False)
sal_df = pd.read_excel(os.path.join(DATASETS_DIR, 'IPL SALARIES 2024.xlsx'))

# %%
# Display columns of the performance DataFrame
print(perf_df.columns)

# %%
# Get unique player names from the salary DataFrame
players = sal_df['Player'].unique()

# %%
# Define Longest Common Subsequence (LCS) score function for string matching
def lcs_score(s1, s2):
    n, m = len(s1), len(s2)
    if n == 0 or m == 0:
        return 0.0

    # Initialize DP table for LCS length calculation
    dp = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if s1[i - 1] == s2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])

    lcs_length = dp[n][m]

    # Calculate LCS score normalized by average length
    average_length = (n + m) / 2.0
    score = lcs_length / average_length
    return score

# %%
# Calculate total runs scored by each striker
runs_dict = perf_df.groupby('Striker')['runs_scored'].sum().to_dict()
strikers = perf_df['Striker'].unique()

# %%
# Match strikers with full player names using LCS score and collect their runs
scored

```

```

matches_by_striker = []
for striker in strikers:
    for player in players:
        score = lcs_score(striker, player)
        if score > 0.80: # Threshold for considering a match
            # Find the actual player name from runs_dict that best matches the
            salary player name
            actual_player_name = max(runs_dict.keys(), key=lambda k:
lcs_score(player, k))
            runs_scored = runs_dict.get(actual_player_name, 0)
            matches_by_striker.append({'Player Name': striker, 'Full Name':
player, 'Runs_Scored': runs_scored, 'LCS Score': score})

# %%
# Create a DataFrame for batsman performance points
ms_df = pd.DataFrame(matches_by_striker)
ms_df['Points for Batsman'] = ms_df['Runs_Scored']
ms_df.to_csv(os.path.join(DATASETS_DIR, 'ms_out.csv'), index=False)

# %%
# Calculate total wickets taken by each bowler
wickets_dict = perf_df.groupby('Bowler')['wicket_confirmation'].sum().to_dict()
bowlers = perf_df['Bowler'].unique()

# %%
# Match bowlers with full player names using LCS score and collect their wickets
taken
matches_by_bowler = []
for bowler in bowlers:
    for player in players:
        score = lcs_score(bowler, player)
        if score > 0.80: # Threshold for considering a match
            # Find the actual player name from wickets_dict that best matches the
            salary player name
            actual_player_name = max(wickets_dict.keys(), key=lambda k:
lcs_score(player, k))
            wickets_taken = wickets_dict.get(actual_player_name, 0)
            matches_by_bowler.append({'Player Name': bowler, 'Full Name': player,
'Wickets_Taken': wickets_taken, 'LCS Score': score})

# %%
# Create a DataFrame for bowler performance points
mb_df = pd.DataFrame(matches_by_bowler)
mb_df['Points for Bowler'] = mb_df['Wickets_Taken'] * 25 # Assuming 25 points per
wicket
mb_df.to_csv(os.path.join(DATASETS_DIR, 'mb_out.csv'), index=False)

# %%
# Merge batsman and bowler performance dataframes on 'Full Name'
combine_df = pd.merge(ms_df, mb_df, on='Full Name', how='outer', suffixes=
('_', 'Batsman'), ('_Bowler'))
print(combine_df.head())

# %%

```

```
# Fill NaN values in 'Points for Batsman' and 'Points for Bowler' with 0
combine_df['Points for Batsman'] = combine_df['Points for Batsman'].fillna(0)
combine_df['Points for Bowler'] = combine_df['Points for Bowler'].fillna(0)

# %%
# Calculate total points by summing batsman and bowler points.
combine_df['Total Points'] = combine_df['Points for Batsman'] + combine_df['Points
for Bowler']
combine_df.to_csv(os.path.join(DATASETS_DIR, 'combined_out.csv'), index=False)

# %%
# Add salary information to the combined DataFrame
combine_df['Salary'] = np.nan
for idx, row in combine_df.iterrows():
    if row['Full Name'] in sal_df['Player'].to_list():
        combine_df.at[idx, 'Salary'] = sal_df[sal_df['Player'] == row['Full
Name']][0]['Rs'].values[0]

# %%
# Display the combined DataFrame with performance points and salary
print(combine_df)

# %%
# Save the combined DataFrame with salary to a CSV file
combine_df.to_csv(os.path.join(DATASETS_DIR, 'combined_out_with_sal.csv'),
index=False)

# %%
# Prepare data for linear regression: 'Total Points' as independent variable (X)
and 'Salary' as dependent variable (y)
optim_agg_combine_df = combine_df[['Full Name', 'Total Points', 'Salary']]
optim_agg_combine_df.to_csv(os.path.join(DATASETS_DIR,
'optim_combine_df_with_sal.csv'), index=False)

y = optim_agg_combine_df['Salary']
X = optim_agg_combine_df[['Total Points']]

# %%
# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
random_state=42)

# %%
# Initialize and fit the Linear Regression model
model = LinearRegression()
model.fit(X_train, y_train)

# %%
# Make predictions and evaluate the model (R-squared and Mean Squared Error)
y_pred = model.predict(X_test)
r2 = r2_score(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)

print(f"R-squared: {r2}")
```

```

print(f"Mean Squared Error: {mse}")

# %%
# Plotting the regression results
plt.scatter(X_test, y_test, color='blue', label='Actual Salary')
plt.plot(X_test, y_pred, color='red', linewidth=2, label='Predicted Salary')
plt.xlabel('Total Points')
plt.ylabel('Salary')
plt.title('Salary vs Total Points')
plt.legend()
plt.show()

```

R Code

```

# Set the base directory for the project
BASE <- "C:\\Users\\ujjwa\\Documents\\VCU\\Pre-
Course\\SCMA632\\Assignments\\A3\\R"
setwd(BASE) # Change working directory to where your data files are located
getwd()     # Confirm working directory

# Define a helper function to install packages if not already installed
install <- function(pkg) {
  if (!require(pkg, character.only = TRUE)) {
    install.packages(pkg, dependencies = TRUE, quiet = TRUE)
  }
}

# Define a helper function to load packages
load <- function(pkg) {
  library(pkg, character.only = TRUE, quietly = TRUE)
}

# Required packages for this analysis
pkgs <- c("tidyverse", "readxl", "car", "stringdist", "lubridate", "stats",
"fitdistrplus")
lapply(pkgs, install)
lapply(pkgs, load)

# --- Part (a): NSS068 Dataset Analysis ---

# Load the NSS068 dataset
df_001 <- read.csv(file.path(BASE, "datasets", "NSS068.csv"))

# Display unique states to verify data loading
print(unique(df_001$state_1))

# Filter data for "MEG" state and select relevant columns for analysis
df_001 <- df_001 %>%
  dplyr::filter(state_1 == "MEG") %>%
  dplyr::select(foodtotal_q, MPCE_MRP, Age, Meals_At_Home, Possess_ration_card,

```

```
Education, No_of_Meals_per_day)

# Display DataFrame information (summary and structure)
str(df_001)
summary(df_001)

# Impute missing values with the mean for specified columns
impute_with_mean <- function(df, cols_to_impute) {
  for (col in cols_to_impute) {
    if (col %in% colnames(df)) {
      df[[col]][is.na(df[[col]])] <- mean(df[[col]], na.rm = TRUE)
    }
  }
  return(df)
}

cols_to_impute <- c('Meals_At_Home', 'No_of_Meals_per_day')
df_001 <- impute_with_mean(df_001, cols_to_impute)

# Replace infinite values (positive or negative) with NA
df_001[sapply(df_001, is.infinite)] <- NA

# Check for any remaining missing values after imputation and replacement
print(colSums(is.na(df_001)))

# Convert 'Possess_ration_card' to factor if it's categorical
df_001$Possess_ration_card <- as.factor(df_001$Possess_ration_card)

# Create and fit the Ordinary Least Squares (OLS) regression model
model_nsso <- lm(foodtotal_q ~ MPCE_MRP + Age + Meals_At_Home +
  Possess_ration_card + Education + No_of_Meals_per_day, data = df_001)

# Print the summary of the regression results
print(summary(model_nsso))

# Calculate Variance Inflation Factor (VIF) to check for multicollinearity
vif_results_nsso <- vif(model_nsso)
print(vif_results_nsso)

# Extract and print the coefficients of the fitted model
coeff_nsso <- coef(model_nsso)
print(coeff_nsso)

# Construct and print the regression equation
eq_nsso <- paste0("y = ", round(coeff_nsso[1], 2))
for (i in 2:length(coeff_nsso)) {
  eq_nsso <- paste0(eq_nsso, " + ", round(coeff_nsso[i], 6), "*",
names(coeff_nsso)[i])
}
print(eq_nsso)

# Delete the DataFrame to free up memory
rm(df_001)
```

```

# --- Part (b): IPL Player Performance and Salary Analysis ---

# Define file paths
perf_path <- file.path(BASE, "datasets", "IPL_ball_by_ball_updated till 2024.csv")
salary_path <- file.path(BASE, "datasets", "IPL SALARIES 2024.xlsx")

# Read the datasets
perf_df <- read_csv(perf_path)
sal_df <- read_excel(salary_path)

# Select only relevant columns and extract the year from the date column
columns_to_select <- c("Match id", "Date", "Season", "Innings No",
                      "Bowler", "Striker", "runs_scored", "wicket_confirmation")
perf_df <- perf_df %>% dplyr::select(all_of(columns_to_select))
# Using dmy() from lubridate for consistent date parsing
perf_df$Year <- year(dmy(perf_df$Date))

# --- Aggregate Data ---

# Summarize total runs by each batsman
runs_agg <- perf_df %>%
  group_by(Year, `Innings No`, Striker) %>%
  summarise(runs_scored = sum(runs_scored), .groups = 'drop')

# Summarize total wickets by each bowler
wickets_agg <- perf_df %>%
  group_by(Year, `Innings No`, Bowler) %>%
  summarise(wicket_confirmation = sum(wicket_confirmation), .groups = 'drop')

# --- Identify Top 3 Performers Each Year ---

years_ipl <- unique(runs_agg$Year)
for (yr in years_ipl) {
  cat("Year:", yr, "\n\nTop 3 Run Scorers:\n")
  print(runs_agg %>% dplyr::filter(Year == yr) %>%
    group_by(Striker) %>%
    summarise(runs = sum(runs_scored), .groups = 'drop') %>%
    arrange(desc(runs)) %>% head(3))

  cat("\n\nTop 3 Wicket Takers:\n")
  print(wickets_agg %>% dplyr::filter(Year == yr) %>%
    group_by(Bowler) %>%
    summarise(wickets = sum(wicket_confirmation), .groups = 'drop') %>%
    arrange(desc(wickets)) %>% head(3))

  cat("\n", strrep("=", 50), "\n\n")
}

# --- Name Matching Between Datasets (Fuzzy Matching) ---

# Using the provided match_names function with jw distance
match_names <- function(name, choices, threshold = 0.2) {
  if (is.na(name)) return(NA)

```

```

dists <- stringdist(name, choices, method = "jw")
min_dist <- min(dists)
if (min_dist <= threshold) return(choices[which.min(dists)])
return(NA)
}

# --- Correlation Between Salary and Performance (2024) ---

runs_2024 <- perf_df %>%
  dplyr::filter(Year == 2024) %>%
  group_by(Striker) %>%
  summarise(runs_scored = sum(runs_scored), .groups = 'drop')

wickets_2024 <- perf_df %>%
  dplyr::filter(Year == 2024) %>%
  group_by(Bowler) %>%
  summarise(wicket_confirmation = sum(wicket_confirmation), .groups = 'drop')

sal_df$Matched_Striker <- sapply(sal_df$Player, match_names, choices =
runs_2024$Striker)
sal_df$Matched_Bowler <- sapply(sal_df$Player, match_names, choices =
wickets_2024$Bowler)

# Merge salary with runs and wickets data for 2024
striker_merged <- merge(sal_df, runs_2024, by.x = "Matched_Striker", by.y =
"Striker", all.x = TRUE)
bowler_merged <- merge(sal_df, wickets_2024, by.x = "Matched_Bowler", by.y =
"Bowler", all.x = TRUE)

# Calculate combined performance points for 2024 for players who have both runs
and wickets
# First, create a base dataframe with unique players and their salaries
combined_perf_2024 <- sal_df %>%
  dplyr::select(Player, Rs) %>%
  dplyr::rename(Full_Name = Player, Salary = Rs)

# Join with runs data
combined_perf_2024 <- combined_perf_2024 %>%
  dplyr::left_join(runs_2024 %>% dplyr::rename(Full_Name = Striker), by =
c("Full_Name")) %>%
  dplyr::rename(Runs_2024 = runs_scored)

# Join with wickets data
combined_perf_2024 <- combined_perf_2024 %>%
  dplyr::left_join(wickets_2024 %>% dplyr::rename(Full_Name = Bowler), by =
c("Full_Name")) %>%
  dplyr::rename(Wickets_2024 = wicket_confirmation)

# Fill NA for runs and wickets with 0 for calculation
combined_perf_2024 <- combined_perf_2024 %>%
  dplyr::mutate(
    Runs_2024 = replace_na(Runs_2024, 0),
    Wickets_2024 = replace_na(Wickets_2024, 0)
  )

```

```

# Calculate Total Points (adjust multiplier as needed, e.g., 25 for wickets)
combined_perf_2024 <- combined_perf_2024 %>%
  dplyr::mutate(Total_Points_2024 = Runs_2024 + (Wickets_2024 * 25))

# Filter for players with valid salary and at least some performance data for
correlation
cor_data_2024 <- combined_perf_2024 %>%
  dplyr::filter(!is.na(Salary) & Total_Points_2024 > 0)

cor_total_points_salary <- cor(cor_data_2024$Salary,
cor_data_2024$Total_Points_2024, use = "complete.obs")

cat("\nCorrelation between Salary and Runs in 2024 (using merged data):",
cor(striker_merged$Rs, striker_merged$runs_scored, use = "complete.obs"))
cat("\nCorrelation between Salary and Wickets in 2024 (using merged data):",
cor(bowler_merged$Rs, bowler_merged$wicket_confirmation, use = "complete.obs"))
cat("\nCorrelation between Salary and Total Points in 2024:",
cor_total_points_salary)

# Prepare data for linear regression: 'Total Points' as independent variable (X)
and 'Salary' as dependent variable (y)
# Using the 2024 combined performance data for this
y_ipl <- cor_data_2024$Salary
X_ipl <- cor_data_2024 %>% dplyr::select(Total_Points_2024)

# Split the data into training and testing sets
set.seed(42) # For reproducibility
train_indices_ipl <- sample(1:nrow(cor_data_2024), size = 0.75 *
nrow(cor_data_2024))
X_train_ipl <- cor_data_2024[train_indices_ipl, 'Total_Points_2024', drop = FALSE]
# Explicitly extract the 'Salary' column as a vector
y_train_ipl <- cor_data_2024[train_indices_ipl, 'Salary'][[1]]
X_test_ipl <- cor_data_2024[-train_indices_ipl, 'Total_Points_2024', drop = FALSE]
# Explicitly extract the 'Salary' column as a vector
y_test_ipl <- cor_data_2024[-train_indices_ipl, 'Salary'][[1]]

# Initialize and fit the Linear Regression model
model_ipl <- lm(Salary ~ Total_Points_2024, data = data.frame(Salary =
y_train_ipl, Total_Points_2024 = X_train_ipl$Total_Points_2024))

# Make predictions
y_pred_ipl <- predict(model_ipl, newdata = data.frame(Total_Points_2024 =
X_test_ipl$Total_Points_2024))

# Evaluate the model (R-squared and Mean Squared Error)
summary_model_ipl <- summary(model_ipl)
r2_ipl <- summary_model_ipl$r.squared
mse_ipl <- mean((y_test_ipl - y_pred_ipl)^2)

print(paste("R-squared (Training):", r2_ipl))
print(paste("Mean Squared Error (Test):", mse_ipl))

# Plotting the regression results

```

```

plot_df <- data.frame(Total_Points_2024 = X_test_ipl$Total_Points_2024,
Actual_Salary = y_test_ipl, Predicted_Salary = y_pred_ipl)

ggplot(plot_df, aes(x = Total_Points_2024)) +
  geom_point(aes(y = Actual_Salary), color = 'blue', alpha = 0.6) + # Removed
  redundant label
  geom_line(aes(y = Predicted_Salary), color = 'red', linewidth = 1) + # Removed
  redundant label
  labs(x = 'Total Points (2024)', y = 'Salary', title = 'Salary vs Total Points in
2024 (IPL)') +
  theme_minimal()

# --- Distribution Fitting for Assigned Player: N Pooran ---

n_pooran_runs <- perf_df %>%
  group_by(Year, Striker, `Match id`) %>%
  summarise(runs_scored = sum(runs_scored), .groups = 'drop') %>%
  dplyr::filter(Striker == "N Pooran") %>%
  pull(runs_scored)

n_pooran_runs_pos <- n_pooran_runs[n_pooran_runs > 0]

fit_norm <- tryCatch(fitdist(n_pooran_runs, "norm"), error = function(e) NULL)
fit_gamma <- tryCatch(fitdist(n_pooran_runs, "gamma"), error = function(e) NULL)
fit_exp <- tryCatch(fitdist(n_pooran_runs, "exp"), error = function(e) NULL)
fit_lnorm <- tryCatch(fitdist(n_pooran_runs_pos, "lnorm"), error = function(e)
NULL)

# Filter out NULL fits for gofstat
fits_pooran <- list(norm = fit_norm, gamma = fit_gamma, exp = fit_exp)
fits_pooran <- Filter(Negate(is.null), fits_pooran)

gof <- if(length(fits_pooran) > 0) tryCatch(gofstat(fits_pooran), error =
function(e) NULL) else NULL
gof_lnorm <- if (!is.null(fit_lnorm) && length(n_pooran_runs_pos) > 0 &&
all(n_pooran_runs_pos > 0)) tryCatch(gofstat(list(fit_lnorm)), error = function(e)
NULL) else NULL

print(gof)
cat("\nLognormal fit (positive data only):\n")
print(gof_lnorm)

# Perform KS test for Gamma fit if it was successful
if (!is.null(fit_gamma)) {
  ks <- ks.test(n_pooran_runs, "pgamma", shape = fit_gamma$estimate["shape"], rate
= fit_gamma$estimate["rate"])
  print(ks)
} else {
  cat("Gamma fit not available for KS test.\n")
}

```

```

# --- Distribution Fitting for Top 3 Batsmen and Bowlers (Last 3 Seasons) ---

# Reusing the provided get_best_fit function
get_best_fit <- function(data) {
  if (length(data) < 2 || all(is.na(data))) return(list(best_dist = "Too few data
or all NA", gof = NULL))
  data_pos <- data[data > 0]
  use_pos_only <- !any(data <= 0) # True if all data is positive

  fits <- list(
    norm = tryCatch(fitdist(data, "norm"), error = function(e) NULL),
    gamma = tryCatch(fitdist(data_pos, "gamma"), error = function(e) NULL), #
Gamma requires positive data
    exp = tryCatch(fitdist(data_pos, "exp"), error = function(e) NULL)      #
Exponential requires positive data
  )

  if (length(data_pos) > 0) {
    fits$lnorm <- tryCatch(fitdist(data_pos, "lnorm"), error = function(e) NULL)
  }

  fits <- Filter(Negate(is.null), fits)

  if (length(fits) == 0) return(list(best_dist = "No suitable fit", gof = NULL))
  if (length(fits) == 1) return(list(best_dist = names(fits), gof = NULL))

  # Use AIC to select the best fit if multiple are available
  aics <- sapply(fits, function(f) f$aic)
  best_dist_name <- names(which.min(aics))

  return(list(best_dist = best_dist_name, gof = NULL)) # Only returning best_dist
as per original structure
}

last_3_seasons <- sort(unique(perf_df$Year), decreasing = TRUE)[1:3]

df_runs_match_level <- perf_df %>%
  group_by(Year, Striker, `Match id`) %>%
  summarise(runs_scored = sum(runs_scored), .groups = 'drop')

df_wickets_match_level <- perf_df %>%
  group_by(Year, Bowler, `Match id`) %>%
  summarise(wicket_confirmation = sum(wicket_confirmation), .groups = 'drop')

for (year in last_3_seasons) {
  cat("\n==== Year:", year, "====")

  top_batsmen <- df_runs_match_level %>%
    dplyr::filter(Year == year) %>%
    group_by(Striker) %>%
    summarise(total_runs = sum(runs_scored), .groups = 'drop') %>%
    arrange(desc(total_runs)) %>%
    slice(1:3) %>% pull(Striker)

```

```
top_bowlers <- df_wickets_match_level %>%
  dplyr::filter(Year == year) %>%
  group_by(Bowler) %>%
  summarise(total_wickets = sum(wicket_confirmation), .groups = 'drop') %>%
  arrange(desc(total_wickets)) %>%
  slice(1:3) %>% pull(Bowler)

cat("\nTop 3 Batsmen Distribution Fits:\n")
for (batsman in top_batsmen) {
  player_data <- df_runs_match_level %>% dplyr::filter(Year == year, Striker ==
batsman) %>% pull(runs_scored)
  result <- get_best_fit(player_data)
  cat("-", batsman, ": Best Fit:", result$best_dist, "\n")
}

cat("\nTop 3 Bowlers Distribution Fits:\n")
for (bowler in top_bowlers) {
  player_data <- df_wickets_match_level %>% dplyr::filter(Year == year, Bowler
== bowler) %>% pull(wicket_confirmation)
  result <- get_best_fit(player_data)
  cat("-", bowler, ": Best Fit:", result$best_dist, "\n")
}
}
```